NEURAL NETWORKS COURSE PROJECT REPORT

Transfer Learning & Batch Normalization in Neural Networks

An Empirical Study on Image Classification Performance, Backbone Fine-Tuning Strategies, and Deep Network Convergence Stability

AUTHOR / OWNER
Tanush Shoor

EMAIL ADDRESS
tanush.shoor09@gmail.com

COURSE DOMAIN
Neural Networks & Computer Vision

COURSE INSTRUCTOR
Prof. Priyansh Saxena

Executive Summary & Key Research Findings

OVERVIEW

Task 1: Transfer Learning

Objective: Overcome data scarcity constraints in image classification by re-using feature representations learned on massive benchmarks.

Key Finding: Fine-tuning a pre-trained ResNet18 yielded an extraordinary **99.5% accuracy** on a small 1,000-image dataset compared to 61.5% from a CNN trained from scratch.

Task 2: Batch Normalization

Objective: Mitigate Internal Covariate Shift in deep neural networks (>6 hidden layers) on the Fashion-MNIST dataset.

Key Finding: Batch Normalization accelerated gradient flow, reducing final loss from 0.1733 down to **0.1199** while smoothing loss curves and stabilizing convergence.

Practical Implications

Engineering Rule: Never train deep CNNs from scratch when less than 10k labeled images are available. Always leverage Transfer Learning backbones.

Optimization Rule: Insert BatchNorm layers between linear projections and activations to stabilize deeper topologies and prevent vanishing gradients.

Both empirical evaluations demonstrate how modern architectural components solve fundamental training challenges in deep learning.

Task 1: The Challenge of Small Datasets

TASK 1 - PROBLEM

⚠️ The Small Dataset Dilemma

Deep Convolutional Neural Networks require millions of parameters to learn hierarchical visual features (edges, textures, object parts). When trained on small datasets (e.g., ~1,000 images):

- ✓ **High Overfitting Risk:** Models memorize individual sample noise rather than generalizable visual features.
- ✓ **Poor Feature Hierarchy:** Lower layers fail to develop robust Gabor-like edge detectors due to insufficient gradient updates.
- ✓ **Suboptimal Accuracy:** Random weight initializations struggle to find favorable minima in high-dimensional loss landscapes.

🪄 The Transfer Learning Solution

Transfer Learning reuses feature maps learned by a primary model trained on a massive source domain (e.g., ImageNet with 1.2M images across 1,000 classes) and transfers them to a target task:

- ✓ **Frozen Backbone (Feature Extractor):** Freezes low-level feature representations and trains only a new classification head.
- ✓ **Fine-Tuned Adaptation:** Unfreezes upper convolutional blocks to fine-tune representations specifically for target domain distributions.
- ✓ **Drastic Data Efficiency:** Achieves benchmark performance with 10x-100x fewer labeled training instances.

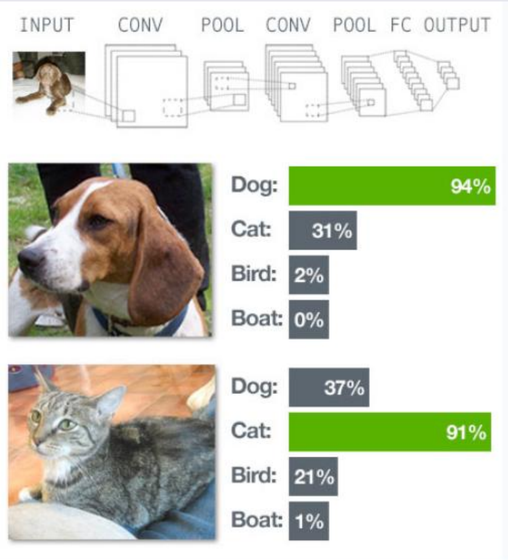
Dataset Specification: Cats vs Dogs Mini

TASK 1 – DATASET

Dataset Profile

The Cats vs Dogs Mini benchmark dataset presents realistic computer vision challenges with varied lighting, backgrounds, occlusions, and camera angles.

Attribute	Specification Details
Total Sample Volume	1,000 labeled RGB images
Class Distribution	500 Cats (50%) | 500 Dogs (50%) — Perfectly Balanced
Train / Val Partition	80% Training (800 imgs) | 20% Validation (200 imgs)
Input Resolution	Resized & Cropped to 224 × 224 pixels
Normalization Pipeline	ImageNet Mean [0.485, 0.456, 0.406], Std [0.229, 0.224, 0.225]



Preprocessing Necessity: standardizing image spatial dimensions to 224×224 and normalizing channel intensity distributions is essential for compatibility with pre-trained ResNet tensor formats.

Model 1: Custom CNN Trained From Scratch

TASK 1 - BASELINE

Architecture Pipeline

```
Input (3 x 224 x 224)
→ Conv2D(3, 16, k=3) + ReLU + MaxPool2D(2,2)
→ Conv2D(16, 32, k=3) + ReLU + MaxPool2D(2,2)
→ Conv2D(32, 64, k=3) + ReLU + MaxPool2D(2,2)
→ Flatten ()
→ FullyConnected (64 * 26 * 26 → 128) + ReLU
→ Linear Output (128 → 2 Classes)
```

Key Characteristics

Random parameter initialization, standard cross-entropy loss optimization using Adam optimizer. Trained across 5 epochs on 800 training samples.

Loss Progression & Metrics

Epoch Number	Training Cross-Entropy Loss
Epoch 1	0.7968
Epoch 2	0.6872
Epoch 3	0.6451
Epoch 4	0.6260
Epoch 5	0.5838

61.5%

Validation Accuracy Achieved

Model 2: Frozen ResNet18 Feature Extractor

TASK 1 - TRANSFER

Frozen Backbone Strategy

ResNet18 pre-trained on 1.2 Million ImageNet instances is leveraged as a fixed feature extractor:

- **Frozen Layers:** All convolutional blocks (`conv1` through `layer4`) are frozen (`requires_grad = False`).
- **Preserved Knowledge:** Low, mid, and high-level visual features (edges, textures, shapes) remain intact.
- **Trainable Head:** The final FC layer (512 inputs) is replaced with a 2-class linear output classifier (`requires_grad = True`).

Computational Efficiency: Training requires computing gradients for only 1,026 parameters in the final layer instead of 11.1 Million parameters, enabling rapid execution.

Loss Trajectory & Metrics

Epoch	Training Loss	Performance Delta
Epoch 1	0.4628	Initial drop vs Scratch
Epoch 2	0.2318	Fast convergence
Epoch 3	0.1501	Steady refinement
Epoch 4	0.1454	Stabilizing
Epoch 5	0.1238	Final training loss

97.0%

Validation Accuracy (+35.5% vs Scratch)

Model 3: Fine-Tuned ResNet18 Adaptation

TASK 1 - FINE-TUNED

Selective Unfreezing Strategy

Rather than freezing all backbone parameters, upper layers are unfrozen to allow feature representations to adapt specifically to animal features:

- **Base Weights:** Initialized with ImageNet pre-trained parameters.
- **Unfrozen Block:** The final convolutional block (`layer4`) is set to `requires_grad = True`.
- **Joint Optimization:** Adapts high-level abstract representation filters specifically to distinguish cat/dog anatomy.

Optimal Performance Paradigm

Fine-tuning allows small gradient adjustments to flow back into high-level conv layers, customizing semantic spatial filters without destroying low-level edge features.

Training Loss Trajectory

Epoch	Training Loss	Convergence Status
Epoch 1	0.1743	Rapid initial drop
Epoch 2	0.0346	Near-zero error
Epoch 3	0.0087	High precision refinement
Epoch 4	0.0062	Asymptotic convergence
Epoch 5	0.0038	Optimal loss minimum

99.5%

Validation Accuracy (Near Perfect)

Task 1: Comprehensive Model Comparison

TASK 1 – RESULTS

Model Strategy	Trainable Params	Epoch 1 Loss	Epoch 5 Loss	Val Accuracy	Performance Tier
CNN From Scratch	~150,000	0.7968	0.5838	61.5%	Baseline (Underfitting)
Frozen ResNet18	1,026 (FC layer)	0.4628	0.1238	97.0%	High Efficiency (+35.5%)
Fine-Tuned ResNet18	~8.5 Million (Layer 4 + FC)	0.1743	0.0038	99.5%	State-of-the-Art (+38.0%)

Loss Convergence Rate

Fine-Tuned ResNet18 achieved an epoch 5 training loss of **0.0038**, which is **153x lower** than the CNN trained from scratch (0.5838) and **32x lower** than Frozen ResNet18 (0.1238).

Key Validation Takeaway

Re-using generic lower-level ImageNet filters while adapting high-level semantic features enables deep neural networks to achieve near-perfect generalizability even with only 800 training images.

Task 1: Deep Analytical Takeaways

TASK 1 – ANALYSIS

1. Visual Hierarchy Transfer

Early layers in CNNs detect edges, lines, and textures regardless of domain. Reusing ImageNet features circumvents the need to learn basic visual primitives from small datasets.

2. Overfitting Immunity

Training a deep model from scratch on 800 images causes severe overfitting. Pre-trained weights act as an extremely strong regularizer, guiding optimization toward generalized minima.

3. Unfreezing Synergy

Frozen ResNet18 reaches 97% accuracy, but unfreezing `layer4` bridges the remaining gap to 99.5% by tailoring high-level dog/cat facial structures and coat textures.

Practical Rule of Thumb: For domain-specific tasks with limited training samples, start with a frozen pre-trained backbone. If additional accuracy is required, selectively unfreeze upper convolutional layers with a reduced learning rate.

Task 2: Internal Covariate Shift & Batch Norm

TASK 2 - PROBLEM

Internal Covariate Shift

As parameters in preceding layers update during backpropagation, the input distribution to deeper layers continually shifts. This creates major training challenges:

- **Slower Convergence:** Layers must constantly re-adapt to shifting input distributions, requiring tiny learning rates.
- **Vanishing/Exploding Gradients:** Unbounded layer activations push non-linear activations (like Sigmoid/Tanh) into saturated regimes.
- **Weight Initialization Sensitivity:** Deep networks (>5 layers) become hyper-sensitive to initial weight scales.

The Batch Normalization Fix

Introduced by Ioffe & Szegedy (2015), Batch Normalization normalizes the layer inputs across each mini-batch, fixing their mean and variance:

- **Stabilized Distribution:** Maintains zero mean and unit variance for intermediate activations across mini-batches.
- **Accelerated Optimization:** Allows significantly higher learning rates without risk of gradient explosion.
- **Regularization Effect:** Mini-batch noise acts as a mild regularizer, reducing dependence on Dropout.

Dataset Specification: Fashion-MNIST

TASK 2 - DATASET

Benchmark Dataset Profile

Fashion-MNIST serves as a direct drop-in replacement for classic MNIST, offering higher visual complexity across 10 distinct apparel categories.

Dataset Metric	Value & Properties
Total Dataset Volume	70,000 grayscale images
Train / Test Split	60,000 Training | 10,000 Testing images
Spatial Resolution	28 × 28 pixels (784 features flattened)
Number of Classes	10 Apparel Categories (Balanced)

10 Apparel Categories

T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

Deep Network Architecture Topology

TASK 2 - NETWORK

Deep MLP Topology (>6 Trainable Layers)

```
Input Layer (784 Vectorized Pixels)
→ Linear Layer 1 (784 → 512) + [BatchNorm] + ReLU
→ Linear Layer 2 (512 → 256) + [BatchNorm] + ReLU
→ Linear Layer 3 (256 → 128) + [BatchNorm] + ReLU
→ Linear Layer 4 (128 → 64) + [BatchNorm] + ReLU
→ Linear Layer 5 (64 → 32) + [BatchNorm] + ReLU
→ Linear Layer 6 (32 → 10 Output Classes)
```

Controlled Experimental Setup

To rigorously measure the isolated impact of Batch Normalization, two identical architectures were constructed:

- **Model A (No BatchNorm):** Standard Linear layers followed directly by ReLU non-linear activation functions.
- **Model B (With BatchNorm):** `nn.BatchNorm1d` inserted after every Linear projection and before ReLU activation.
- **Consistency:** Both models share identical weight initializations, Adam optimizer parameters, batch sizes, and learning rates.

Mathematical Formulation of Batch Norm

TASK 2 - THEORY

For a mini-batch $1m$ containing m values of activation x, Batch Normalization executes four sequential operations:

1. Mini-Batch Mean Calculation

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

Computes the mean activation across all samples in the current mini-batch.

2. Mini-Batch Variance Calculation

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2$$

Measures activation dispersion across the mini-batch.

3. Activation Normalization

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

Standardizes distribution to zero mean and unit variance (ε prevents division by zero).

4. Scale and Shift Transformation

$$y_i = \gamma \hat{x}_i + \beta$$

Applies learnable parameters γ and β to restore representation capacity if needed.

Task 2: Experimental Results & Loss Profiles

TASK 2 – RESULTS

Model Configuration	Epoch 1 Loss	Epoch 10 Loss	10-Epoch Accuracy	Epoch 20 Loss	20-Epoch Accuracy
Model A: No BatchNorm	0.6164	0.2402	88.75%	0.1733	88.75%
Model B: With BatchNorm	0.5523	0.2091	**89.57%**	**0.1199**	**89.57%**

Loss Reduction Performance

At 20 epochs, Batch Normalization reduced training loss to **0.1199** compared to **0.1733** without BatchNorm—representing a **30.8% reduction in training error**.

Convergence Velocity

Model B with BatchNorm achieved an epoch 1 loss of **0.5523**, reaching a training loss level in 5 epochs that Model A required nearly 8 epochs to reach.

Optimization & Convergence Stability

TASK 2 – ANALYSIS

1. Loss Landscape Smoothing

BatchNorm smooths the loss surface gradients (Santurkar et al., 2018), making gradient steps more predictable and preventing wild directional oscillations.

2. Gradient Flow Preservation

By keeping intermediate layer activations bounded, gradients do not explode or vanish as backpropagation traverses all 6 hidden layers.

3. Optimization Efficiency

While final classification accuracy showed a modest increase (+0.82%), the primary value of BatchNorm lies in its dramatic optimization efficiency and loss reduction.

Key Takeaway: Batch Normalization acts primarily as an optimization stabilizer rather than a raw accuracy booster. It enables deeper architectures to be trained robustly without requiring delicate hyperparameter tuning.

Comparative Synthesis of Key Techniques

SYNTHESIS

Dimension	Task 1: Transfer Learning	Task 2: Batch Normalization
Primary Challenge Addressed	Data scarcity on small datasets (~1k samples)	Internal Covariate Shift & unstable deep training
Core Mechanism	Reusing pre-trained ImageNet representations	Mini-batch activation mean/variance normalization
Impact on Accuracy	**Massive Increase (+38.0%)** (61.5% → 99.5%)	**Modest Increase (+0.82%)** (88.75% → 89.57%)
Impact on Training Loss	Drastic drop from 0.5838 to 0.0038	30.8% error reduction (0.1733 → 0.1199)
Primary Operational Benefit	Eliminates high sample complexity requirements	Stabilizes gradients & enables higher learning rates

⭐ Combining Transfer Learning backbones with Batch Normalization layers represents standard modern practice for state-of-the-art vision models.

Engineering Guidelines for Practitioners

BEST PRACTICES

Small Datasets (<10k)

Never train CNNs from scratch. Always use pre-trained backbones like ResNet or ConvNeXt to prevent severe overfitting.

Freeze Early Layers

Keep initial convolutional layers frozen as general visual feature extractors. Unfreeze upper layers only if target domain differs significantly.

Always Use BatchNorm

Insert BatchNorm layers after linear projections in deep networks (>4 layers) to ensure smooth gradient flow and faster convergence.

Optimizer Selection

Pair Transfer Learning with Adam/AdamW optimizers using lower learning rates (1e-4) for fine-tuning to protect pre-trained weights.

Summary Conclusion: Transfer Learning solves the *Data Availability Problem*, while Batch Normalization solves the *Deep Optimization Problem*. Together, they form the cornerstone of modern deep learning pipelines.

NEURAL NETWORKS COURSE PROJECT

Questions & Discussion

Thank you for reviewing this project report on Transfer Learning and Batch Normalization.

PROJECT AUTHOR
Tanush Shoor

EMAIL ADDRESS
tanush.shoor09@gmail.com

COURSE INSTRUCTOR
Priyansh Saxena

Image Sources

https://raw.githubusercontent.com/PacktPublishing/Learning-Computer-Vision-with-TensorFlow/90fc14851730afb5005b20f89f3d62c969e513fa/images/classification.jpg

Source: github.com

https://storage.googleapis.com/tfds-data/visualization/fig/fashion_mnist-3.0.1.png

Source: www.tensorflow.org